

PostgreSQL Optimization Tips (A–Z)

A practical, battle-tested checklist for tuning PostgreSQL systems in production. Focus on impact first, measure everything, and let the optimizer work for you.

A —

- Analyze queries with EXPLAIN and EXPLAIN ANALYZE.
- Avoid guessing—let execution plans show real bottlenecks.

B —

- Build the right indexes for WHERE, JOIN, and ORDER BY clauses.
- Balance B-Tree, GIN, and BRIN indexes based on access patterns.

C —

- Choose CTEs for clarity; PostgreSQL 12+ can inline them.
- Confirm planner behavior with EXPLAIN.

D —

- Design schemas deliberately—data model mistakes are expensive.
- Default to BIGINT for primary keys and TEXT over VARCHAR(n).

E —

- Eliminate unused indexes and dead columns.
- Each index adds write and vacuum overhead.

F —

- Filter early—push selective WHERE conditions down.
- Reduce row counts before joins whenever possible.

G —

- Gather statistics regularly using ANALYZE.
- Good stats equal good plans.

H —

- Handle hot tables carefully—expect contention.
- Use partitioning or batching for heavy write paths.

I —

- Index only what you read.
- More indexes $\neq$ better performance.

J —

- Join on indexed, compatible data types.
- Watch join order and row explosion in execution plans.

K —

- Keep transactions short.
- Long-lived transactions block vacuum and bloat tables.

L —

- Limit result sets aggressively.
- OFFSET is expensive; prefer keyset pagination.

M —

- Monitor slow queries using `pg_stat_statements`.
- Measure before and after every change.

N —

- Normalize by default, denormalize when proven necessary.
- Base decisions on query patterns, not theory.

O —

- Optimize for the common case.
- Focus on the 5% of queries causing 95% of load.

P —

- Partition large tables with clear pruning keys.
- Ensure queries actually benefit from partition pruning.

Q —

- Question sequential scans—but don't fear them blindly.
- They can be optimal for large result sets.

R —

- Right-size `work_mem` and `shared_buffers`.
- Avoid global over-allocation that causes memory pressure.

S —

- Select only the columns you need.
- Wide rows slow everything down.

T —

- Tune autovacuum for your workload.
- Default settings are rarely optimal at scale.

U —

- Use JSONB sparingly and with intention.
- Index JSON paths explicitly when queried.

V —

- Vacuum aggressively enough to prevent bloat.
- Bloat silently kills performance over time.

W —

- Watch for write amplification from indexes and triggers.
- Every write has a cost.

X —

- eXplain everything during incidents.
- Execution plans tell the real story.

Y —

- Yield to the optimizer—don't fight it without proof.
- Planner hints are rarely the right first move.

Z —

- Zero in on slowest, highest-cost queries first.
- Optimize what matters, ignore the rest.